



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

Instituto de Ciencias Básicas e Ingeniería

Licenciatura en Ciencias Computacionales

Reporte de Práctica / Proyecto Final Construcción y Validación de un Tokenizador para Java

Alumno: Daniela Lopez Martinez

Catedrático: EDUARDO CORNEJO VELÁZQUEZ

Grupo: 6-3

Asignatura: AUTOMÁTAS Y COMPILADORES



1. Fundamentos Teóricos

El escaneo léxico constituye la etapa preliminar durante la traducción de programas. Su propósito fundamental radica en inspeccionar la cadena de entrada fuente carácter por carácter para agruparlos en unidades con significado sintáctico denominadas **tokens**.

A fin de establecer la taxonomía conceptual del compilador, se diferencian tres términos esenciales:

- **Token:** Categoría o etiqueta abstracta que identifica la clase gramatical del componente (por ejemplo, `KEYWORD`, `IDENTIFIER`, `OPERATOR`).
- **Lexema:** La muestra o subcadena concreta extraída del texto plano que encaja con una regla (por ejemplo, `if`, `"suma"`, `"3.14"`).
- **Patrón:** La especificación formal formalizada mediante Expresiones Regulares (ER) que norma la estructura que debe poseer un lexema para asignarle un token específico.

2. Metas de la Práctica

- Formular patrones con Expresiones Regulares para discriminar cada uno de los elementos léxicos de la especificación Java.
- Programar un motor de extracción usando la clase `Matcher` y patrones con grupos nombrados en Java.
- Implementar un mecanismo tolerante a fallos (*Panic Recovery*) que localice fallas sin abortar la lectura del archivo.
- Generar métricas de procesamiento consolidando tablas de distribución estadística y exportación CSV.

3. Arquitectura del Analizador

La tokenización opera mediante un patrón único consolidado (*Master Regex*) estructurado en varios bloques:

A. Palabras Reservadas e Identificadores

Se empleó el delimitador de frontera `\b` alrededor de la lista de términos reservados. Con esto se evita que un identificador compuesto (como `intCounter`) se interprete de forma errónea como el tipo de dato `int`.

B. Operadores y Prioridades

Para evitar ambigüedades con el principio de coincidencia más larga (*Maximal Munch*), las combinaciones multicarácter (`++`, `+=`, `==`, `!=`) preceden a los símbolos individuales (`+`, `=`).

C. Literales y Constantes

Se incluyeron patrones específicos para capturar literales flotantes, notación científica, enteros en base decimal, binaria (`0b`), octal (`0`) y hexadecimal (`0x`), así como cadenas de caracteres con caracteres de escape.

D. Recuperación de Errores

Si la entrada contiene un símbolo no reconocido (ej. `@`), la rutina `reportErrors` registra la anomalía con su coordenada (L, C), omite el símbolo no válido y continúa el procesamiento.

4. Código Fuente (JavaLexer.java)

```
1 import java.util.regex.*;
2 import java.util.*;
3 import java.util.stream.*;
4
5 public class JavaLexer {
6
7     enum TokenType {
8         JAVADOC, BLOCKCOMMENT, LINECOMMENT,
9         KEYWORD, IDENTIFIER,
10        OPERATOR, SEPARATOR,
11        FLOATLIT, HEXADECIMALLIT, BINARYLIT, OCTALLIT, INTEGERLIT
12    },
13    STRINGLIT, CHARLIT,
14    WHITESPACE, UNKNOWN
15
16    record Token(TokenType type, String lexeme, int line, int
column) {
17        public String toString() {
18            return String.format("[L%-3d C%-3d] %-14s -> \"%s\"",
19                line, column, type, lexeme);
20        }
21        public String toCsv() {
22            String lex = lexeme.replace("\"", "\\\"");
23            return String.format("%d,%d,%s,\"%s\"", line, column,
type, lex);
24        }
25    }
26
27    public static class LexicalError extends RuntimeException {
28        private final int line, column;
29        private final char illegal;
30
31        public LexicalError(char c, int line, int col) {
32            super(String.format(
33                "Error lexico [linea %d, col %d]: caracter
34                ilegal '%c' (U+%04X)",
35                line, col, c, (int) c));
36            this.line = line; this.column = col; this.illegal = c
;
37        }
38        public int getLine() { return line; }
39        public int getColumn() { return column; }
40        public char getIllegal(){ return illegal; }
41    }
42
43    static final String KW_PATTERN =
    "\\b(abstract|assert|boolean|break|byte|case|catch|
char|" +
```

```

44         "class|const|continue|default|do|double|else|enum|
extends|" +
45         "final|finally|float|for|if|implements|import|
instanceof|" +
46         "int|interface|long|new|package|private|protected|
public|" +
47         "return|short|static|super|switch|synchronized|this|
throw|" +
48         "throws|try|void|volatile|while|true|false|null)\b";
49
50     static final Pattern MASTER = Pattern.compile(
51         "(?<JAVADOC>/\s*\s*[\s\S]*?\s*/)|" +
52         "(?<BLOCKCOMMENT>/\s*[\s\S]*?\s*/)|" +
53         "(?<LINECOMMENT>//[^\n]*)|" +
54         "(?<STRINGLIT>\"([^\\"\\\\]|\\\\\\\\|\\\\\\.|\\\\\\\\\\\\\\\\)*\\")|" +
55         "(?<CHARLIT>'([^\''\\\\]|\\\\\\\\(u[0-9a-fA-F]{4}|.))')|" +
56         "(?<FLOATLIT>[0-9][0-9_]*\.[0-9][0-9_]*([eE
] [+ -]?[0-9]+)?[fFdD]?)|" +
57         "(?<HEXADECIMALLIT>0[xX][0-9a-fA-F][0-9a-fA-F_]*[1L
]?)|" +
58         "(?<BINARYLIT>0[bB][01][01_]*[1L]?)|" +
59         "(?<OCTALLIT>0[0-7]+[1L]?)|" +
60         "(?<INTEGERLIT>0|[1-9][0-9_]*[1L]?)|" +
61         "(?<KEYWORD>" + KW_PATTERN + ")|" +
62         "(?<IDENTIFIER>[a-zA-Z_$][a-zA-Z0-9_$]*)|" +
63         "(?<OPERATOR>" +
64         "==|!=|<|=|&&|\\|\\|\\|<=>|>=>|<<|>>|>>|" +
65         "\\+\\+|\\-\\-|\\+|=|\\-|=|\\*=|/=|%=|&|=|\\|=|\\^=|" +
66         "[+\\-*/%<=>!&|^~?:]" +
67         ")|" +
68         "(?<SEPARATOR>[( ){}\\[\\];,\\.])|" +
69         "(?<WHITESPACE>[ \\t\\r\\n]+)",
70         Pattern.MULTILINE
71     );
72
73     public static List<Token> tokenize(String source, List<
LexicalError> errors) {
74         List<Token> tokens = new ArrayList<>();
75         Matcher m = MASTER.matcher(source);
76         int line = 1, pos = 0;
77
78         while (m.find()) {
79             if (m.start() > pos) {
80                 reportErrors(source, pos, m.start(), line, errors
);
81                 line += countNewlines(source.substring(pos, m.
start()));
82             }
83
84             if (m.group("WHITESPACE") != null ||
85                 m.group("JAVADOC") != null ||

```

```

86         m.group("BLOCKCOMMENT") != null) {
87             line += countNewlines(m.group());
88         }
89
90         if (m.group("KEYWORD") != null) {
91             tokens.add(mk(TokenType.KEYWORD, m, source, line)
92 );
93         } else if (m.group("IDENTIFIER") != null) {
94             tokens.add(mk(TokenType.IDENTIFIER, m, source,
95 line));
96         } else if (m.group("OPERATOR") != null) {
97             tokens.add(mk(TokenType.OPERATOR, m, source, line
98 ));
99         } else if (m.group("SEPARATOR") != null) {
100             tokens.add(mk(TokenType.SEPARATOR, m, source,
101 line));
102         } else if (m.group("FLOATLIT") != null) {
103             tokens.add(mk(TokenType.FLOATLIT, m, source, line
104 ));
105         } else if (m.group("HEXADECIMALLIT") != null) {
106             tokens.add(mk(TokenType.HEXADECIMALLIT, m, source
107 , line));
108         } else if (m.group("BINARYLIT") != null) {
109             tokens.add(mk(TokenType.BINARYLIT, m, source,
110 line));
111         } else if (m.group("OCTALLIT") != null) {
112             tokens.add(mk(TokenType.OCTALLIT, m, source, line
113 ));
114         } else if (m.group("INTEGERLIT") != null) {
115             tokens.add(mk(TokenType.INTEGERLIT, m, source,
116 line));
117         } else if (m.group("STRINGLIT") != null) {
118             tokens.add(mk(TokenType.STRINGLIT, m, source,
119 line));
120         } else if (m.group("CHARLIT") != null) {
121             tokens.add(mk(TokenType.CHARLIT, m, source, line)
122 );
123         }
124
125         pos = m.end();
126     }
127
128     if (pos < source.length()) {
129         reportErrors(source, pos, source.length(), line,
130 errors);
131     }
132
133     return tokens;
134 }

```

```

124     private static Token mk(TokenType t, Matcher m, String src,
125     int line) {
126         return new Token(t, m.group(), line, colOf(src, m.start()
127     ));
128     }
129
130     private static long countNewlines(String s) {
131         return s.chars().filter(c -> c == '\n').count();
132     }
133
134     private static int colOf(String source, int pos) {
135         int nl = source.lastIndexOf('\n', pos - 1);
136         return pos - nl;
137     }
138
139     private static void reportErrors(String source, int start,
140     int end,
141                                     int line, List<LexicalError>
142     errors) {
143         int col = colOf(source, start);
144         for (int i = start; i < end; i++) {
145             char c = source.charAt(i);
146             LexicalError err = new LexicalError(c, line, col + (i
147             - start));
148             errors.add(err);
149             System.err.println("ALERT: " + err.getMessage());
150         }
151     }
152
153     public static Map<TokenType, Long> frequency(List<Token>
154     tokens) {
155         return tokens.stream().collect(Collectors.groupingBy(
156             Token::type, TreeMap::new, Collectors.counting())
157     );
158     }
159
160     public static String toCsv(List<Token> tokens) {
161         StringBuilder sb = new StringBuilder("linea,columna,tipos,
162     lexema\n");
163         for (Token t : tokens) sb.append(t.toCsv()).append('\n');
164         return sb.toString();
165     }
166
167     public static void main(String[] args) {
168         String codigoPrueba = ""
169         public class Prueba {
170             int x = 10;
171             double pi = 3.1416;
172             String msg = "Hola mundo";
173             int hex = 0xFF;
174             @

```

```

167         if (x >= 10) {
168             x++;
169         }
170     }
171     """;
172
173     List<LexicalError> errores = new ArrayList<>();
174     List<Token> tokens = tokenize(codigoPrueba, errores);
175
176     System.out.println("=== TOKENS ===");
177     tokens.forEach(System.out::println);
178
179     System.out.println("\n=== ERRORES L XICOS ===");
180     if (errores.isEmpty()) {
181         System.out.println("No se encontraron errores
182 l xicos.");
183     } else {
184         errores.forEach(e -> System.out.println(e.getMessage
185 ()));
186     }
187
188     System.out.println("\n=== FRECUENCIA ===");
189     frequency(tokens).forEach((k, v) -> System.out.println(k
190 + ": " + v));
191
192     System.out.println("\n=== CSV ===");
193     System.out.println(toCsv(tokens));
194 }
195 }

```